SMART BATHROOM PRESENCE SENSOR (REDWAN-BATH-NODE)

A-TO-Z ENGINEERING REFERENCE, HARDWARE MANUAL & PRODUCTION CODE

1. Complete Hardware Module BOM & Bangladeshi Market Alternatives

To build a **10–20 year lifespan** presence detection node that withstands humid bathroom environments and continuous mains switching, use the primary industrial components below or their validated Bangladeshi market drop-in alternatives.

Primary Component	Engineering Function	Validated Local Market Alternative	Key Technical Notes for 10–20 Year Uptime PDF
HLK-PM01 (5V 3W)	Isolated AC-to-DC Mains Power Supply	HLK-5M05 (5V 5W), Hi-Link HLK-10M05, or Mean Well IRM-03-5	Fully encapsulated switch-mode power supply; isolates 220V AC mains from low-voltage DC.
ESP32-C3 SuperMini	Core Microcontroller / Radio Node	ESP32-C3 Zero, ESP32 DevKitC WROOM-32	Compact RISC-V microcontroller with built-in Wi-Fi and Bluetooth LE.
HLK-LD2410B (24GHz)	FMCW mmWave Human Presence Sensor	HLK-LD2410C or HLK-LD2420	Detects both moving targets and micro-motion (static chest breathing).
5V Relay Module (Active-LOW)	Switches 220V AC Lighting Load	G3MB-202P 5V Solid State Relay (SSR)	**SSR is superior:** Solid-state relays switch silently with zero mechanical

			contacts to wear out.
1A Glass Fuse & PCB Holder	Mains Overcurrent & Short-Circuit Protection	1A 250V Radial Micro Fuse (392 Series) or 1A PPTC	Prevents catastrophic electrical failure on the primary AC line.
10D471K Varistor (MOV)	Mains High-Voltage Surge Clamping	7D471K or 14D471K MOV	Clamps high-voltage grid transients above 470 V.
$10\,\Omega$ 2W Resistor	Power-Up Inrush Current Limiter	10D-9 NTC Thermistor or $10\,\Omega$ 2W Flameproof Resistor	Protects the HLK-PM01 input stage during cold power-up.
$0.1\,\mu\text{F}$ 275VAC X2 Cap	RC Snubber Contact Arc Suppression	100 nF / Code 104 275V–310V AC X2 Safety Film Cap	**Must be X2 safety-rated;** suppresses inductive kickback across relay terminals.
$100\,\Omega$ 2W Resistor	RC Snubber Series Current Limiter	$100\,\Omega$ 2W Metal Film / Metal Oxide Resistor	Wired in series with the X2 capacitor across relay COM & N.O.
$220\,\mu\text{F}$ 16V Capacitor	5V DC Rail RF Pulse Filter	$330\,\mu\text{F}$ 16V or $470\,\mu\text{F}$ 16V Electrolytic Cap	Absorbs 24 GHz radar transmitter current spikes to prevent ESP32 brownouts.

2. System Architecture & Dual-Zone Circuit Schematic

To prevent surface arcing and moisture-induced current tracking in humid bathroom environments, the PCB layout is strictly partitioned into **Zone A (High-Voltage AC Mains)** and **Zone B (Low-Voltage DC Logic)**. The two zones are physically separated by a **2 mm**

air-gap isolation slot cut through the fiberglass substrate.

Plaintext

```
=====================================================================================
ZONE A: HIGH-VOLTAGE AC MAINS (220V AC)
=====================================================================================

 220V Live (L) ──── [ 1A Fast Fuse ] ───┬──── [ 10Ω 2W Inrush Limiter ] ────> [ HLK-PM01 AC PIN (L) ]
                                        │
                                        ├─── [ 10D471K Varistor (MOV) ] ───┐  (Surge Clamp)
                                        │                                  │
 220V Neutral (N) ─────────────────────────────┼────────────────────────────────────┼───> [ HLK-PM01 AC PIN (N) ]
                                        │                                  │
                                        ├─── [ 0.1µF X2 Cap ]─[ 100Ω 2W ]──┘  (RC Snubber Protection)
                                        │       │        │         │
                                        │       └────────┴─────────┘
                                        │            │             │
 To Bathroom Light <─── [ RELAY N.O. ] ─┴──────────────────────────┴─── [ RELAY COM ]

=====================================================================================
 [ PCB CREEPAGE AIR GAP: 2MM PHYSICAL ISOLATION SLOT CUT THROUGH FIBERGLASS SUBSTRATE ]
=====================================================================================
ZONE B: LOW-VOLTAGE DC LOGIC (5V / 3.3V)
=====================================================================================

                                        +───────────────────────────────────────> Relay Module VCC (+)
                                        │
                                        ├──────────────────────────────────────>
```

[redacted]

[redacted]

Why the Physical Isolation Slot is Critical

- **Surface Creepage in Humid Air:** High humidity forms an invisible film of condensation and dust across the PCB. Over time, 220 V AC can crawl across this damp surface toward low-voltage DC tracks, creating short circuits and fire hazards.
- **The Air-Gap Solution:** Open air is a superior electrical insulator compared to a damp PCB surface. Cutting a 2 mm slot completely through the fiberglass between Zone A and Zone B breaks the moisture tracking path.

3. Complete Hardware Pinout & Interconnection Table

Sub-Assembl	Pin /	ESP32-C3	Power Rail / PCB Track	Engineering Purpose &

y	Terminal	Pin	Connection	Description PDF
HLK-PM01 Module	AC Input L	—	Fused 220V Live Line	Mains AC input (via 1A Fuse & Inrush Limiter).
HLK-PM01 Module	AC Input N	—	220V Neutral Line	Mains AC return line.
HLK-PM01 Module	DC Output +VO	5V / VIN	5V DC Power Rail	Regulated low-voltage DC power rail.
HLK-PM01 Module	DC Output -VO	GND	Common Ground Rail	Common system ground reference.
HLK-LD2410 B Radar	VCC	—	5V DC Power Rail	5 V input for 24 GHz RF oscillator.
HLK-LD2410 B Radar	GND	—	Common Ground Rail	Common ground reference.
HLK-LD2410 B Radar	OUT	GPIO 8	Logic Input Track	Sends digital HIGH (3.3 V) on human presence.
5V Relay Module	VCC	—	5V DC Power Rail	Power supply for relay optocoupler/coil.

5V Relay Module	GND	—	Common Ground Rail	Common ground reference.
5V Relay Module	IN	GPIO 10	Logic Output Track	Active-LOW trigger (LOW = ON, HIGH = OFF).
5V Relay Module	COM	—	Fused 220V Live Line	AC grid input to relay contact.
5V Relay Module	N.O.	—	Bathroom Light (Live)	Normally Open contact; closes to illuminate load.
$220\,\mu\text{F}$ 16V Cap	Positive (+)	—	5V DC Power Rail	Absorbs 24 GHz RF spikes to prevent MCU reset.
$220\,\mu\text{F}$ 16V Cap	Negative (-)	—	Common Ground Rail	Filter capacitor ground return.
RC Snubber Circuit	$0.1\,\mu\text{F} + 100\,\Omega$	—	Across COM & N.O.	Suppresses inductive arcing across contacts.

4. Required Arduino Libraries & Environment Setup

To compile the firmware without errors in Arduino IDE or PlatformIO, ensure the ESP32 Core and required libraries are installed:

1. Board Manager Configuration

- **Platform:** esp32 by Espressif Systems (Version 3.x.x recommended).
- **Board Selection:** ESP32C3 Dev Module or ESP32-C3 SuperMini.
- **USB CDC On Boot:** Enabled (Required for Serial Monitor output on USB-C ESP32-C3 boards).
- **Flash Size / Partition Scheme:** 4MB (32Mb) / Default 4MB with spiffs (1.2MB APP/1.5MB SPIFFS).

2. Required Library Dependencies

- **WiFi.h**: Native ESP32 Wi-Fi station connectivity.
- **PubSubClient.h**: MQTT client by Nick O'Leary (install via Arduino Library Manager).
- **esp_now.h**: Native Expressif peer-to-peer low-latency wireless protocol.
- **ArduinoOTA.h**: Native Over-The-Air wireless firmware updates.

5. Production Firmware (redwan-bath-node.ino)

This production-ready C++ firmware incorporates:

- **Active-LOW Relay Control:** Accurately drives standard opto-isolated relay modules (LOW = ON, HIGH = OFF).
- **Dual-Stack Wireless Redundancy:** Transmits status updates concurrently over **ESP-NOW** (sub-10 ms local RF to Master Hub) and **MQTT** (to Home Assistant).
- **Asynchronous Network Maintenance:** Reconnection loops run non-blocking in the background so local presence switching never hangs.
- **OTA Capability:** Secure Over-The-Air updates enabled at 192.168.x.x (Redwan-Bathroom-Node / abcd2005-).

```
C++
#include <WiFi.h>
#include <PubSubClient.h>
#include <esp_now.h>
#include <ArduinoOTA.h>

// ==========================================
// 1. PIN ASSIGNMENTS & HARDWARE LOGIC
// ==========================================
#define RADAR_OUT_PIN  8    // HLK-LD2410B OUT pin
#define RELAY_PIN      10   // Active-LOW Relay pin[cite: 2, 3, 4]
#define RELAY_ON       LOW  // Active-LOW Relay Module (LOW = ON, HIGH = OFF)
#define RELAY_OFF      HIGH

// ==========================================
// 2. NETWORK & MQTT CREDENTIALS
// ==========================================
const char* WIFI_SSID     = "Syndicate";
```

```
const char* WIFI_PASSWORD = "586792023-";
const char* MQTT_SERVER   = "192.168.0.40";  // Raspberry Pi HA Broker IP
const int   MQTT_PORT     = 1883;
const char* MQTT_USER     = "redwanmqtt";
const char* MQTT_PASS     = "abcd2005-";

// Master Center Hub MAC Address for ESP-NOW Peer-to-Peer Backup
uint8_t masterMacAddress[] = { 0x68, 0xFE, 0x71, 0x8B, 0x58, 0x50 };

WiFiClient espClient;
PubSubClient mqttClient(espClient);

enum OperatingMode {
  AUTO_MODE,
  FORCE_ON,
  FORCE_OFF
};
OperatingMode currentMode = AUTO_MODE;

bool lastRelayState = false;
unsigned long lastPresenceTimestamp = 0;
const unsigned long OFF_TIMEOUT_MS = 3000; // 3-second empty room turn-off delay[cite: 2, 3,
4]
unsigned long lastNetworkRetry = 0;

typedef struct struct_message {
  char nodeID[20];
  bool lightActive;
  uint8_t mode; // 0=Auto, 1=ForceON, 2=ForceOFF
} struct_message;
struct_message telemetryPacket;

typedef struct cmd_message {
  uint8_t targetMode; // 0=Auto, 1=ForceON, 2=ForceOFF
} cmd_message;
cmd_message incomingCmd;

// ==========================================
// 3. TELEMETRY & NETWORK BROADCAST
// ==========================================
void transmitStatus(bool state) {
  telemetryPacket.lightActive = state;
  telemetryPacket.mode = (uint8_t)currentMode;
```

```
 // 1. Send low-latency ESP-NOW RF telemetry to Master Center Hub
 esp_now_send(masterMacAddress, (uint8_t*)&telemetryPacket, sizeof(telemetryPacket));

 // 2. Publish MQTT state to Home Assistant Broker
 if (mqttClient.connected()) {
   mqttClient.publish("home/bathroom/light/state", state ? "ON" : "OFF", true);
   const char* modeStr = (currentMode == AUTO_MODE) ? "AUTO" : ((currentMode ==
FORCE_ON) ? "ON" : "OFF");
   mqttClient.publish("home/bathroom/light/mode_state", modeStr, true);
 }
}

// ==========================================
// 4. ESP-NOW COMMAND RECEIVER (RainMaker App Control)
// ==========================================
void onESPNowCommandRecv(const esp_now_recv_info* recv_info, const uint8_t* incomingData,
int len) {
 memcpy(&incomingCmd, incomingData, sizeof(incomingCmd));

 if (incomingCmd.targetMode == 1) {
   currentMode = FORCE_ON;
   Serial.println("[ESP-NOW CMD] Overrode Light -> FORCE ON");
 } else if (incomingCmd.targetMode == 2) {
   currentMode = FORCE_OFF;
   Serial.println("[ESP-NOW CMD] Overrode Light -> FORCE OFF");
 } else if (incomingCmd.targetMode == 0) {
   currentMode = AUTO_MODE;
   Serial.println("[ESP-NOW CMD] Restored Light -> AUTO MODE");
   // Immediately evaluate radar when returning to Auto mode
   bool currentPresence = (digitalRead(RADAR_OUT_PIN) == HIGH);
   digitalWrite(RELAY_PIN, currentPresence ? RELAY_ON : RELAY_OFF);
   lastRelayState = currentPresence;
   if (currentPresence) lastPresenceTimestamp = millis();
 }
 transmitStatus(lastRelayState);
}

// ==========================================
// 5. MQTT RECEIVER CALLBACK (HA Manual Override)
// ==========================================
void handleMQTTMessage(char* topic, byte* payload, unsigned int length) {
 String message = "";
```

```
for (unsigned int i = 0; i < length; i++) {
  message += (char)payload[i];
}

if (String(topic) == "home/bathroom/light/mode_cmd") {
  if (message == "AUTO") {
    currentMode = AUTO_MODE;
    Serial.println("[OVERRIDE] Mode set to AUTO -> Re-evaluating Radar");
    bool currentPresence = (digitalRead(RADAR_OUT_PIN) == HIGH);
    digitalWrite(RELAY_PIN, currentPresence ? RELAY_ON : RELAY_OFF);
    lastRelayState = currentPresence;
    if (currentPresence) lastPresenceTimestamp = millis();
  } else if (message == "ON") {
    currentMode = FORCE_ON;
    Serial.println("[OVERRIDE] Mode set to FORCE ON");
  } else if (message == "OFF") {
    currentMode = FORCE_OFF;
    Serial.println("[OVERRIDE] Mode set to FORCE OFF");
  }
  transmitStatus(lastRelayState);
}
}

// ============================================
// 6. SETUP ROUTINE
// ============================================
void setup() {
 Serial.begin(115200);
 delay(300);

 pinMode(RELAY_PIN, OUTPUT);
 digitalWrite(RELAY_PIN, RELAY_OFF);
 lastRelayState = false;

 pinMode(RADAR_OUT_PIN, INPUT_PULLDOWN);

 WiFi.mode(WIFI_STA);
 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

 if (esp_now_init() == ESP_OK) {
  esp_now_register_recv_cb(onESPNowCommandRecv);
  esp_now_peer_info_t peerInfo = {};
  memcpy(peerInfo.peer_addr, masterMacAddress, 6);
```

```
  peerInfo.channel = 0;
  peerInfo.encrypt = false;
  esp_now_add_peer(&peerInfo);
}

 mqttClient.setServer(MQTT_SERVER, MQTT_PORT);
 mqttClient.setCallback(handleMQTTMessage);

 ArduinoOTA.setHostname("Redwan-Bathroom-Node");
 ArduinoOTA.setPassword("abcd2005-");
 ArduinoOTA.begin();

 strcpy(telemetryPacket.nodeID, "REDWAN-BATH-NODE");
 Serial.println("[BOOT] REDWAN-BATH-NODE Online!");
}

// ===========================================
// 7. MAIN PRODUCTION LOOP
// ===========================================
void loop() {
 // 1. STATE MACHINE & CONTINUOUS RELAY ENFORCEMENT
 if (currentMode == AUTO_MODE) {
  bool currentPresence = (digitalRead(RADAR_OUT_PIN) == HIGH);

  if (currentPresence) {
   lastPresenceTimestamp = millis();
   if (!lastRelayState) {
    digitalWrite(RELAY_PIN, RELAY_ON);
    lastRelayState = true;
    transmitStatus(true);
    Serial.println("[RADAR] Occupied -> Light ON");
   }
  } else {
   if (lastRelayState && (millis() - lastPresenceTimestamp >= OFF_TIMEOUT_MS)) {
    digitalWrite(RELAY_PIN, RELAY_OFF);
    lastRelayState = false;
    transmitStatus(false);
    Serial.println("[RADAR] Clear -> Light OFF");
   }
  }
 }
 else if (currentMode == FORCE_ON) {
  // Continually enforce physical ON state so it NEVER fails to switch
```

```
    digitalWrite(RELAY_PIN, RELAY_ON);
    if (!lastRelayState) {
      lastRelayState = true;
      transmitStatus(true);
    }
  }
  else if (currentMode == FORCE_OFF) {
    // Continually enforce physical OFF state
    digitalWrite(RELAY_PIN, RELAY_OFF);
    if (lastRelayState) {
      lastRelayState = false;
      transmitStatus(false);
    }
  }

  // 2. NETWORK & OTA MAINTENANCE
  if (WiFi.status() == WL_CONNECTED) {
    ArduinoOTA.handle();

    if (!mqttClient.connected()) {
      if (millis() - lastNetworkRetry > 5000) {
        lastNetworkRetry = millis();
        if (mqttClient.connect("REDWAN-BATH-NODE", MQTT_USER, MQTT_PASS)) {
          mqttClient.subscribe("home/bathroom/light/mode_cmd");
          transmitStatus(lastRelayState);
        }
      }
    } else {
      mqttClient.loop();
    }
  }

  delay(10);
}
```

6. HLKRadarTool App & Distance Gate Calibration Blueprint

To prevent false triggers from people walking down the hallway while maintaining sensitivity to chest breathing when sitting still on the toilet, configure the **HLK-LD2410B** distance gates using the **HLKRadarTool** mobile app.

1. Understanding Distance Gates

The LD2410B divides its detection cone into 9 Distance Gates (Gate 0 through Gate 8). Each gate represents roughly 0.75 meters (75 cm) of linear depth:

- **Gate 0 ($0.00\text{ m} - 0.75\text{ m}$):** Doorway Threshold
- **Gate 1 ($0.75\text{ m} - 1.50\text{ m}$):** Vanity / Sink Area
- **Gate 2 ($1.50\text{ m} - 2.25\text{ m}$):** Toilet Area *(Maximum sensitivity required for micro-motion!)*
- **Gate 3 ($2.25\text{ m} - 3.00\text{ m}$):** Shower / Far Wall
- **Gate 4+ ($>3.00\text{ m}$):** Outside Bathroom *(Must be set to 0 sensitivity to prevent hallway false triggers)*

Plaintext

[redacted]

2. Step-by-Step Calibration Blueprint

1. **Set Maximum Gate Range:** Measure the physical depth of your bathroom. For a 2.2 meter bathroom, set **Max Moving Gate = 3** and **Max Static Gate = 3**. This mathematically restricts radar depth to $3 \times 0.75\text{ m} = 2.25\text{ m}$, blinding the sensor to hallway traffic.
2. **Tune Dynamic vs. Static Sensitivities:**
   - **Gate 0 & 1 (Doorway & Vanity):** Set Moving Sensitivity = 60, Static Sensitivity = 40.
   - **Gate 2 (Toilet / Primary Still Zone):** Set Moving Sensitivity = 50, Static Sensitivity = 15 (or 20). *(Note: In HLKRadarTool, a **lower number** means **higher sensitivity** to subtle chest movements)*[cite: 4].
   - **Gate 3 through 8 (Outside Zone):** Set Moving Sensitivity = 0, Static Sensitivity = 0.
3. **Set Unmanned Duration:** Set **Unmanned Duration = 5 seconds** in the app settings[cite: 2, 3, 4]. This acts as a hardware-level debounce window before the OUT pin drops LOW, preventing relay chattering.

7. Step-by-Step Bench Testing & Zero PCB Fabrication Guide

A. Bench-Test Verification Setup (Low-Voltage Only)

Before soldering high-voltage AC mains lines, bench-test the DC logic on your desk:

1. Connect 5V, GND, GPIO 8 (Radar OUT), and GPIO 10 (simulated LED/relay).
2. Upload the sketch and open Arduino Serial Monitor at 115200 baud.
3. **Verify Presence ON:** Wave your hand in front of the sensor; GPIO 10 must trigger immediately (LOW for active-LOW relay) and output [RADAR] Occupied -> Light ON.
4. **Verify 3-Second Delay OFF:** Leave the sensor's field of view. Once the LD2410B's internal 5-second unmanned timer drops OUT to LOW, watch the ESP32 count exactly **3000 ms (3 seconds)** before turning the relay OFF.

B. Permanent Zero PCB Fabrication Steps

1. **Cut the Isolation Slot:** Allocate 40% of the board area to Zone A (AC) and 60% to Zone B (DC). Cut a 2 mm physical slot through the fiberglass along the dividing line using a rotary tool.
2. **Assemble High-Voltage Zone A:**
   - Solder the 1A Glass Fuse Holder on the 220V Live input terminal.
   - Solder the 10D471K MOV in parallel across Live and Neutral.
   - Solder the $10\,\Omega$ 2W Inrush Limiter resistor in series between the fuse and HLK-PM01 AC-L pin.
   - Solder the RC Snubber ($0.1\,\mu\text{F}$ X2 cap + $100\,\Omega$ 2W resistor in series) directly across the relay COM and N.O. terminals.
   - **Reinforce AC Traces:** Solder a strand of 1.5 mm^2 copper wire along the bottom AC Live and Neutral tracks to handle peak current loads safely.
3. **Assemble Low-Voltage Zone B:**
   - Solder female header sockets for the ESP32-C3 SuperMini.
   - Solder the **$220\,\mu\text{F}$ 16V capacitor** directly across the 5V and GND output pins of the HLK-PM01.
   - Strip and solder the SH1.27mm radar wires directly into the PCB tracks *(Do not use spring breadboard headers for permanent builds)*.
4. **Moistureproofing & Sealing:**
   - Clean the soldered PCB bottom with Isopropyl Alcohol (IPA).
   - Apply 2–3 coats of clear **Acrylic Conformal Coating** over the soldered tracks. *(Warning: Never apply coating over the ESP32 antenna, LD2410B radar antenna face, or glass fuse)*.
   - Mount inside an IP65/IP67 waterproof ABS junction box high on the bathroom wall.

8. A-to-Z Debugging & Troubleshooting Checklist

Plaintext

[redacted]

Complete Troubleshooting Reference Table

Symptom / Error	Root Engineering Cause	Calibrated Hardware / Firmware Solution
ESP32-C3 Bootloops (SPI_FAST_FLASH_BOOT / POWERON_RESET)	Voltage sag on the V_{IN} (5 V) line caused by thin jumper wires or RF current spikes when the LD2410B oscillator fires.	Solder a **$220\,\mu\text{F}$ to $470\,\mu\text{F}$ 16V electrolytic capacitor** directly across the 5 V and GND pins on the PCB. Solder wires directly instead of DuPont connectors.
Relay Clicks	High capacitive inrush	Verify the

Continuously or Arcs with LED Bulb	current from the LED bulb's internal driver ballasts.	**$0.1\,\mu\text{F}$ 275VAC X2 safety film capacitor** and **$100\,\Omega$ 2W resistor** are soldered in series across relay COM and N.O..
Light Turns ON When Someone Walks in Hallway	Radar microwave penetration through open doors or thin gypsum/brick walls.	In the HLKRadarTool app, reduce **Max Moving Gate** and **Max Static Gate** to **3** (2.25 m depth).
Light Turns OFF While Sitting Still on Toilet	Static sensitivity threshold is too high to detect subtle chest breathing.	In HLKRadarTool, change **Gate 2 Static Sensitivity** from default down to **15** or **20**. Verify **Unmanned Duration = 5s**[cite: 3, 4].
Wireless OTA Code Update Fails After Box Sealing	Incorrect mDNS hostname configuration or router firewall blocking local OTA ports.	In Arduino IDE, navigate to **Tools > Port > Network ports** and select **Redwan-Bathroom-Node at 192.168.x.x**[cite: 3, 4]. Verify router permits peer-to-peer LAN Wi-Fi.

9. Exporting to a Standalone Reference PDF

To convert this reference manual into a standalone PDF file for your workshop build binder:

1. **In VS Code / Cursor / Markdown Editors:** Open the Markdown Preview tab (Ctrl + Shift + V on Windows/Linux or Cmd + Shift + V on macOS) $\rightarrow$ Right-click inside the preview area $\rightarrow$ Select **Print...** $\rightarrow$ Choose **Save as PDF**.
2. **In Web Browsers (Chrome / Edge):** Press **Ctrl + P** (or **Cmd + P**) $\rightarrow$ Set **Destination** to **Save as PDF** $\rightarrow$ Uncheck **Headers and footers** $\rightarrow$ Set **Margins** to **Default** $\rightarrow$ Click **Save**.